Numerical Linear Algebra Final Project

Multigrid and Uzawa-Type Methods for the MAC Discretization of the Stokes Equations

Yuhan Ye
School of Mathematical Sciences, Peking University

February 2023

Abstract

We consider the steady Stokes equations on the unit square and their MAC (marker-and-cell) discretization on a staggered grid, which yields a large sparse saddle-point linear system. Three iterative solvers are implemented and compared: (i) a V-cycle multigrid method with the distributive Gauss–Seidel (DGS) smoother, (ii) the Uzawa iteration with conjugate-gradient (CG) solves of the velocity subproblem, and (iii) the Inexact Uzawa iteration where each velocity subproblem is solved by a V-cycle preconditioned CG (PCG) method. Numerical experiments confirm second-order accuracy of the MAC scheme and show that multigrid-based approaches achieve essentially grid-independent convergence.

Keywords. V-cycle multigrid, DGS smoother, Uzawa iteration, Inexact Uzawa iteration, conjugate gradient (CG), preconditioned conjugate gradient (PCG).

1 Model problem and project tasks

1.1 Continuous model

Let $\Omega = (0,1)^2$. The (steady) Stokes equations are

$$\begin{cases} -\Delta \boldsymbol{u} + \nabla p = \boldsymbol{F}, & (x,y) \in \Omega, \\ \nabla \cdot \boldsymbol{u} = 0, & (x,y) \in \Omega, \end{cases} \tag{1}$$

where $\boldsymbol{u} = (u,v)^\top$ is the velocity, p is the pressure, and $\boldsymbol{F} = (f,g)^\top$ is the forcing. The boundary conditions are mixed Dirichlet–Neumann conditions componentwise:

$$\frac{\partial u}{\partial n} = b \text{ on } y = 0, \qquad \frac{\partial u}{\partial n} = t \text{ on } y = 1, \qquad u = 0 \text{ on } x = 0, 1, \tag{2}$$

$$\frac{\partial v}{\partial n} = l \text{ on } x = 0, \qquad \frac{\partial v}{\partial n} = r \text{ on } x = 1, \qquad v = 0 \text{ on } y = 0, 1, \tag{3}$$

where n is the outward unit normal.

1.2 Manufactured solution

The numerical test uses the exact solution

$$\begin{aligned} u(x,y) &= \big(1 - \cos(2\pi x)\big)\sin(2\pi y), \\ v(x,y) &= -\big(1 - \cos(2\pi y)\big)\sin(2\pi x), \\ p(x,y) &= \frac{x^3}{3} - \frac{1}{12}. \end{aligned} \tag{4}$$

Substituting (4) into (1) gives the forcing

$$\begin{aligned} f(x,y) &= -4\pi^2(2\cos(2\pi x)-1)\sin(2\pi y) + x^2, \\ g(x,y) &= \quad 4\pi^2(2\cos(2\pi y)-1)\sin(2\pi x). \end{aligned} \tag{5}$$

The boundary data b, t, l, r are obtained by differentiating (4) in the normal direction on the corresponding boundary segments.

1.3 Discrete error metric

For $h = 1/N$, the discrete velocity error is measured by

$$\begin{aligned} e_N = h\Bigg(&\sum_{j=1}^{N}\sum_{i=1}^{N-1} \left|u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}})\right|^2 \\ &+ \sum_{j=1}^{N-1}\sum_{i=1}^{N} \left|v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j)\right|^2 \Bigg)^{1/2}, \end{aligned} \tag{6}$$

where $x_i = ih$, $y_j = jh$, $x_{i-\frac{1}{2}} = (i-\frac{1}{2})h$, and $y_{j-\frac{1}{2}} = (j-\frac{1}{2})h$.

1.4 Project tasks and stopping criterion

We implement and compare three solvers for the MAC discretization of (1)–(3):

(1) a V-cycle multigrid method with the DGS smoother; we report baseline performance and study the effect of the smoothing parameters $\nu_1 = \nu_2$;

(2) the Uzawa iteration with CG solves of the velocity subproblem; we study the influence of the step size α;

(3) the Inexact Uzawa iteration, where the velocity subproblem is solved by PCG preconditioned by a (nearly symmetric) V-cycle multigrid method; we study the influence of the inner tolerance τ.

Unless otherwise stated, all *outer* iterations are terminated when

$$\frac{\|r_k\|_2}{\|r_0\|_2} \le 10^{-8}. \tag{7}$$

2 MAC discretization and the saddle-point linear system

2.1 Staggered unknowns

Let $h = 1/N$. The MAC staggered grid stores

$$u_{i,j-\frac{1}{2}} \approx u(x_i, y_{j-\frac{1}{2}}), \quad 1 \le i \le N-1,\ 1 \le j \le N,$$

$$v_{i-\frac{1}{2},j} \approx v(x_{i-\frac{1}{2}}, y_j), \quad 1 \le i \le N,\ 1 \le j \le N-1,$$

$$p_{i-\frac{1}{2},j-\frac{1}{2}} \approx p(x_{i-\frac{1}{2}}, y_{j-\frac{1}{2}}), \quad 1 \le i, j \le N.$$

Thus u is located on vertical cell faces, v on horizontal cell faces, and p at cell centers.

2.2 Discrete operators (interior stencils)

On interior points, the momentum equations are discretized by second-order finite differences. For example, for u one obtains

$$-(\Delta_h u)_{i,j-\frac{1}{2}} + (\partial_x^h p)_{i,j-\frac{1}{2}} = f_{i,j-\frac{1}{2}}, \tag{8}$$

where

$$(\Delta_h u)_{i,j-\frac{1}{2}} = \frac{u_{i+1,j-\frac{1}{2}} - 2u_{i,j-\frac{1}{2}} + u_{i-1,j-\frac{1}{2}}}{h^2} + \frac{u_{i,j+\frac{1}{2}} - 2u_{i,j-\frac{1}{2}} + u_{i,j-\frac{3}{2}}}{h^2},$$
$$(\partial_x^h p)_{i,j-\frac{1}{2}} = \frac{p_{i+\frac{1}{2},j-\frac{1}{2}} - p_{i-\frac{1}{2},j-\frac{1}{2}}}{h}.$$

The formula for v is analogous, with

$$(\partial_y^h p)_{i-\frac{1}{2},j} = \frac{p_{i-\frac{1}{2},j+\frac{1}{2}} - p_{i-\frac{1}{2},j-\frac{1}{2}}}{h}.$$

Boundary conditions are imposed by standard ghost-point elimination, which modifies the second differences near Neumann boundaries.

2.3 Discrete incompressibility

The discrete divergence is imposed at each cell center:

$$(\nabla_h \cdot \boldsymbol{u})_{i-\frac{1}{2},j-\frac{1}{2}} := \frac{u_{i,j-\frac{1}{2}} - u_{i-1,j-\frac{1}{2}}}{h} + \frac{v_{i-\frac{1}{2},j} - v_{i-\frac{1}{2},j-1}}{h} = 0, \qquad 1 \le i, j \le N. \tag{9}$$

2.4 Block form and pressure normalization

Collect all velocity unknowns in U and all pressure unknowns in P. The MAC discretization yields the saddle-point system

$$\begin{pmatrix} A & B \\ B^\top & 0 \end{pmatrix} \begin{pmatrix} U \\ P \end{pmatrix} = \begin{pmatrix} F \\ 0 \end{pmatrix}. \tag{10}$$

Here A corresponds to the discrete vector Laplacian (including boundary conditions), B is the discrete gradient, and $B^\top$ is the discrete divergence. The residual is defined by

$$r := \begin{pmatrix} F - AU - BP \\ -B^\top U \end{pmatrix},$$

and $\|\cdot\|_2$ denotes the Euclidean norm.

Pressure normalization. The discrete pressure is determined only up to an additive constant. In the implementation we fix the gauge by enforcing a zero-mean pressure: after each outer iteration, we replace P by $P - \bar{p}\,\mathbf{1}$, where $\bar{p}$ is the average of all pressure entries.

Remark (SPD property of A). Under the imposed mixed boundary conditions, the discrete velocity operator A is symmetric positive definite on the velocity unknowns, so (P)CG methods are applicable for the velocity subproblem.

3 Problem 1: V-cycle multigrid with DGS smoothing

3.1 DGS smoother

A key difficulty of (10) is the incompressibility constraint. The distributive Gauss–Seidel (DGS) smoother is designed for MAC Stokes systems and consists of two stages:

(i) **Momentum relaxation.** Apply one Gauss–Seidel sweep to the momentum equations with the pressure kept fixed. For an interior u-unknown,

$$\begin{aligned} u_{i,j-\frac{1}{2}} \leftarrow \frac{1}{4}\Big(h^2 f_{i,j-\frac{1}{2}} + u_{i-1,j-\frac{1}{2}} + u_{i+1,j-\frac{1}{2}} + u_{i,j-\frac{3}{2}} + u_{i,j+\frac{1}{2}} \\ - h\big(p_{i+\frac{1}{2},j-\frac{1}{2}} - p_{i-\frac{1}{2},j-\frac{1}{2}}\big)\Big), \end{aligned} \tag{11}$$

and v is updated analogously.

(ii) **Local divergence correction.** After restriction/prolongation in multigrid, coarse-grid residual equations may have a nonzero divergence right-hand side. We therefore write the constraint on each level as

$$B^\top U = D, \tag{12}$$

where $D = 0$ for the original finest-grid problem but $D \neq 0$ may occur for residual equations on coarse levels. Define the divergence defect at cell (i, j) by

$$r_{i,j} := D_{i,j} - (\nabla_h \cdot \boldsymbol{u})_{i-\frac{1}{2},j-\frac{1}{2}}. \tag{13}$$

For an interior cell, set

$$\delta = \frac{h}{4} r_{i,j}. \tag{14}$$

Then update the four surrounding face velocities by

$$\begin{aligned} u_{i,j-\frac{1}{2}} &\leftarrow u_{i,j-\frac{1}{2}} + \delta, \quad u_{i-1,j-\frac{1}{2}} \leftarrow u_{i-1,j-\frac{1}{2}} - \delta, \\ v_{i-\frac{1}{2},j} &\leftarrow v_{i-\frac{1}{2},j} + \delta, \quad v_{i-\frac{1}{2},j-1} \leftarrow v_{i-\frac{1}{2},j-1} - \delta. \end{aligned} \tag{15}$$

This changes the discrete divergence at cell (i, j) by $4\delta/h = r_{i,j}$ and hence eliminates the defect locally. Near boundaries the same idea applies, with the factor $1/4$ replaced by $1/3$ or $1/2$ depending on the number of available neighboring faces. A local pressure correction is applied simultaneously (standard DGS) to improve the coupling between momentum and incompressibility.

3.2 V-cycle structure and implementation notes

Let N be the finest-grid resolution and let L denote the resolution on the coarsest level (so coarsening proceeds by $N \mapsto N/2 \mapsto \cdots \mapsto L$). A standard V-cycle consists of:

(1) ν_1 pre-smoothing DGS steps;

(2) residual computation and restriction to the next coarser grid;

(3) recursive coarse-grid correction;

(4) prolongation of the correction and update on the fine grid;

(5) ν_2 post-smoothing DGS steps.

Implementation details:

- **Matrix-free stencils.** All operator applications are computed by stencil operations without assembling the full matrices.
- **Coarsest level.** Since the coarsest problem is small, it is solved by 100 DGS iterations when $L \in \{2, 4, 8\}$.
- **Coarse-grid operator.** On each level, the MAC discretization is applied directly to define (A_h, B_h) (rediscretization).
- **Transfer operators.** The restriction/prolongation operators recommended in the project description are used.

3.3 Numerical results (baseline parameters)

We first use the baseline choice $\nu_1 = \nu_2 = 2$ and $L = 4$. Table 1 reports the number of outer V-cycles, CPU time, and the error e_N.

Table 1: V-cycle multigrid with DGS smoothing ($\nu_1 = \nu_2 = 2$, $L = 4$).

N	#cycles	CPU time (s)	e_N
64	7	0.0257	1.5000e-03
128	7	0.0497	3.7363e-04
256	7	0.2097	9.3398e-05
512	7	1.3178	2.3349e-05
1024	7	12.5019	5.8367e-06
2048	6	88.0770	1.4533e-06
4096	6	928.0110	3.6066e-07

The error decreases by approximately a factor of 4 when h is halved, consistent with second-order accuracy. Moreover, the number of outer V-cycles is essentially independent of N, which is the expected optimal-order multigrid behavior.

3.4 Influence of ν_1 and ν_2

Because pre- and post-smoothing play symmetric roles here, we test $\nu_1 = \nu_2$ and focus on $N \in \{512, 1024\}$ with $L = 4$.

Table 2: Effect of $\nu_1 = \nu_2$ for $N = 512$ ($L = 4$).

$\nu_1 = \nu_2$	#cycles	CPU time (s)	e_N
1	12	1.7768	2.3349e-05
2	7	1.3314	2.3349e-05
3	6	1.2563	2.3348e-05
4	5	1.2753	2.3340e-05
5	5	1.4620	2.3346e-05
6	5	1.5597	2.3348e-05
10	4	1.7913	2.3333e-05

Table 3: Effect of $\nu_1 = \nu_2$ for $N = 1024$ ($L = 4$).

$\nu_1 = \nu_2$	#cycles	CPU time (s)	e_N
1	12	23.8155	5.8372e-06
2	7	16.4871	5.8367e-06
3	6	17.0450	5.8363e-06
4	5	14.9919	5.8282e-06
5	5	16.8083	5.8339e-06
6	5	18.6067	5.8357e-06
10	4	18.1219	5.8138e-06

Increasing $\nu_1 = \nu_2$ reduces the number of outer cycles, but each cycle becomes more expensive. In these experiments, $\nu_1 = \nu_2 \in \{2, 3, 4\}$ provides the best time-to-solution.

3.5 A Jacobi-type variant (comparison)

To quantify the importance of Gauss–Seidel ordering in the smoother, we also test a Jacobi-type variant where the momentum relaxation in (11) is performed simultaneously (Jacobi update), while keeping the same multigrid framework.

Table 4: Jacobi-type smoother within the same V-cycle framework ($L = 4$).

N	#cycles	CPU time (s)	e_N
256	10	0.9229	9.3398e-05
512	10	3.3729	2.3349e-05
1024	11	27.3283	5.8372e-06
2048	11	388.1940	1.7202e-06

The Jacobi-type smoother requires more V-cycles and significantly more time than DGS, consistent with the fact that Gauss–Seidel is a stronger smoother for elliptic operators.

3.6 Remark on performance bottlenecks

Profiling indicates that for large N (e.g., $N = 1024$ and $N = 2048$) the DGS sweeps dominate the runtime due to large nested loops in `MATLAB`. A potential optimization is to rewrite stencil applications using vectorized operations to reduce loop overhead.

4 Problem 2: Uzawa iteration with CG velocity solves

4.1 Uzawa iteration

Starting from (10), the Uzawa iteration updates

$$\begin{aligned} AU^{k+1} &= F - BP^k, \\ P^{k+1} &= P^k + \alpha\, B^\top U^{k+1}, \end{aligned} \tag{16}$$

where $\alpha > 0$ is the pressure step size. This is a Richardson iteration for the Schur complement equation $B^\top A^{-1} BP = B^\top A^{-1} F$. Unless otherwise stated, we use $\alpha = 1$.

4.2 CG stopping rule (inner solve)

Direct factorization is infeasible for large N, so the velocity subproblem is solved by CG in a matrix-free manner. In the *accurate* variant, CG is terminated when

$$\|r\|_2 \le \varepsilon_{\text{acc}} \|F - BP^k\|_2, \qquad \varepsilon_{\text{acc}} = 10^{-10}, \tag{17}$$

where $r = (F - BP^k) - AU$ is the inner residual.

4.3 Numerical results (accurate CG solves)

Table 5: Uzawa iteration with accurate CG solves ($\varepsilon_{\text{acc}} = 10^{-10}$). The last column reports "CG iters in outer 1 / outer 2".

N	#Uzawa steps	CPU time (s)	e_N	CG iters
64	2	1.0428	1.5000e-03	1727 / 1687
128	2	6.7195	3.7363e-04	3566 / 3437
256	2	96.6829	9.3398e-05	7169 / 7074
512	2	1552.2400	2.3349e-05	–

Although only two outer Uzawa steps are needed, the method is not competitive because each accurate CG solve becomes very expensive as N increases.

4.4 Influence of α

For $N = 256$, we compare several values of α.

Table 6: Effect of the Uzawa step size α for $N = 256$ ($\varepsilon_{\text{acc}} = 10^{-10}$).

α	#Uzawa steps	CPU time (s)	e_N	CG iters
0.5	2	106.2530	9.3398e-05	7169 / 7074
1.0	2	96.6829	9.3398e-05	7169 / 7074
1.5	2	143.9800	9.3398e-05	7169 / 7074

Among these choices, $\alpha = 1$ yields the smallest runtime while maintaining the same accuracy.

4.5 A practical variant: inexact CG solves

A practical improvement is to solve the velocity equation only to a looser tolerance and to warm-start CG with the previous velocity iterate. This can substantially reduce the inner CG iterations while still achieving the same discretization accuracy in e_N.

Table 7: Practical Uzawa implementation with warm-started (inexact) CG solves.

N	#Uzawa steps	CPU time (s)	e_N	CG iters
64	2	0.0819	1.5000e-03	196 / 146
128	2	0.6578	3.7363e-04	393 / 282
256	2	6.6162	9.3398e-05	781 / 546
512	2	111.1576	2.3349e-05	1548 / 1047

Compared with Table 5, the speedup is substantial, indicating that overly accurate inner solves are unnecessary for the outer Uzawa iteration.

5 Problem 3: Inexact Uzawa with V-cycle preconditioned CG

5.1 Method description

The Inexact Uzawa iteration uses the same pressure update as (16), but replaces the velocity solve by a preconditioned CG method. At each outer step, PCG is applied to

$$AU^{k+1} = F - BP^k, \tag{18}$$

with a multigrid V-cycle as the preconditioner, followed by the pressure update

$$P^{k+1} = P^k + \alpha\, B^\top U^{k+1}.$$

Unless otherwise stated, we use $\alpha = 1$.

5.2 Symmetric multigrid preconditioner

PCG requires a symmetric positive definite (SPD) preconditioner. To make the multigrid preconditioner as close to symmetric as possible, we use *symmetric* Gauss–Seidel (one forward sweep followed by one backward sweep) as the smoother in the V-cycle. In practice, PCG converges robustly with this choice.

5.3 Inner stopping rule and parameters

The method depends on

- α: pressure step size;
- τ: relative PCG tolerance for the velocity subproblem;
- ν_1, ν_2, L: multigrid parameters in the V-cycle preconditioner;
- k_0: a prescribed PCG iteration count for the very first outer step.

In the inner PCG solve of (18), we stop when

$$\frac{\|r^{(m)}\|_2}{\|r^{(0)}\|_2} \le \tau,$$

where $r^{(m)}$ is the PCG residual after m iterations. For the first outer iteration we run exactly k_0 PCG steps and then switch to the above stopping rule.

5.4 Numerical results (baseline parameters)

We use the baseline choice $L = 4$, $\nu_1 = \nu_2 = 2$, $\alpha = 1$, $\tau = 10^{-3}$, and $k_0 = 2$.

Table 8: Inexact Uzawa with V-cycle PCG ($L = 4$, $\nu_1 = \nu_2 = 2$, $\alpha = 1$, $\tau = 10^{-3}$, $k_0 = 2$). The last column reports "PCG iters in outer 1 / 2 / 3".

N	#outer steps	CPU time (s)	e_N	PCG iters
64	3	0.0278	1.5000e-03	2 / 3 / 3
128	3	0.0601	3.7363e-04	2 / 3 / 3
256	3	0.2630	9.3399e-05	2 / 3 / 3
512	3	1.4914	2.3349e-05	2 / 3 / 3
1024	3	14.3476	5.8373e-06	2 / 3 / 3
2048	3	148.6886	1.4595e-06	2 / 3 / 3

With only 2–3 PCG iterations per outer step, the overall performance is comparable to the multigrid solver in Problem 1 and is far better than plain Uzawa with accurate CG solves.

5.5 Influence of τ

Fix $N = 1024$ and the other baseline parameters. Table 9 shows how τ affects convergence and runtime.

Table 9: Effect of the PCG tolerance τ for $N = 1024$.

τ	#outer steps	CPU time (s)	e_N
1	diverges	–	–
1e-1	4	12.6432	5.8372e-06
1e-2	4	15.6832	5.8372e-06
1e-3	3	14.3476	5.8373e-06
1e-4	3	13.0225	5.8372e-06
1e-5	3	14.0797	5.8372e-06
1e-20	3	34.9568	5.8372e-06

If τ is too large, the velocity subproblem is under-solved and the outer iteration may fail to converge. Once τ is sufficiently small, the method is robust and the runtime is fairly insensitive to τ; choosing τ excessively small only wastes inner iterations without improving the final discretization error.

6 Conclusions

(1) The DGS-smoothed V-cycle multigrid solver achieves essentially grid-independent convergence and delivers second-order accurate solutions for the MAC discretization.

(2) The plain Uzawa iteration converges in very few outer steps, but its efficiency is dictated by how the velocity subproblem is solved. Accurate CG solves are prohibitively expensive for large N.

(3) The Inexact Uzawa method with a (nearly symmetric) multigrid preconditioner is efficient: only a few PCG steps are required per outer iteration, and the overall performance is comparable to a standalone multigrid solve.